TensorForge 2.0 — Phase 2 report draft

Submission status: local prototype; official schema validation, public hosting, container execution, and video recording are pending. This draft must be updated with final deployment and submission evidence.

1. Problem and solution

RideEat support messages arrive through chat, email, and call transcripts in English, Sinhala, Tamil, transliterated text, and mixed scripts. The prototype predicts a primary issue category, optional secondary issue, urgency, and support team. A browser dashboard supports individual triage and JSON batches; the service also exposes synchronous and asynchronous evaluation APIs.

Classification runs locally using trained TF-IDF and logistic-regression pipelines. No LLM, remote model, model download, or paid inference API is used at runtime. The evaluation API key authenticates requests to our own service; it is not a model-provider credential.

2. Architecture and design decisions

FastAPI serves the dashboard, public `/health`, and protected prediction/job endpoints. The configured `API_KEY` is compared with the supplied `X-API-Key` or bearer token using a constant-time comparison. A missing configuration denies protected access. The public demo dashboard uses a same-origin server bridge that supplies the configured key internally. No key is sent to the browser, and no key-entry prompt is shown. Direct prediction/job endpoints continue to require the exact key. Deployment must preserve the judges' exact supplied key throughout evaluation.

The service validates requests before inference, preserves batch ordering, and echoes supplied ticket IDs. Synchronous batches support 1–100 tickets. The async job implementation accepts 1–5,000 tickets, including the required evaluation range above 2,000; the organizer's actual contract must determine whether a stricter minimum is required. CPU-bound synchronous inference runs in a thread pool so it does not directly block the HTTP event loop.

SQLite persists job state and results. One worker processes jobs in chunks of 100; at most four jobs are active or queued. The API supports status polling, pageable results, idempotency keys, cancellation/discard, and 24-hour retention checked when results are accessed. Interrupted jobs are marked failed after a restart. Run one API process per job store and mount persistent storage for hosting. Queue limits return a retryable response rather than allowing unbounded job submissions.

The container copies a pre-trained artifact and installs dependencies at build time. Its startup command serves port 8000. Runtime inference requires no internet. A Docker build/run without network has not yet been verified because the local Docker daemon is unavailable.

3. Training and development

The supplied dataset has 4,000 training examples and 800 validation examples. Input features include channel, subject, and message text; ticket IDs and supplied language labels are excluded. Text is normalized to Unicode NFC. Word 1–2 grams capture lexical phrases while character n-grams support scripts, spelling variants, and transliteration.

Three pipelines learn primary category, secondary category (including a no-secondary class), and urgency. The current primary pipeline uses boundary-aware character 2–6 grams, balanced class weights, and logistic-regression regularization `C=40`. Secondary and urgency pipelines use character 2–5 grams and `C=3`. Logistic regression uses a fixed seed of 42. Urgency and secondary probability thresholds are 0.2 and 0.1 respectively. Narrow precedence rules handle safety, delay/refund, and missing-item/refund conflicts. Routing follows the application's fixed category-to-team map, which still requires confirmation against organizer schemas.

The current configuration incorporates feature, weighting, threshold, and precedence choices. No saved historical ablation results are available, so this report does not claim a measured gain over an earlier version. Reproduction: `python scripts/train_model.py` trains and saves the artifact and overall metrics; `python scripts/evaluate_model.py` evaluates the saved artifact and records category, language, and confusion breakdowns.

Validation limitation: the 800-example validation split informed model and threshold selection. These are development scores, not independent test results. The judges' unseen holdout dataset will provide the independent evaluation. Dataset notes also caution that Sinhala/Tamil synthetic content has not received independent native-speaker review.

4. Evaluation and findings

Current artifact: `tf-phase2-baseline-1.2.1`.

Output	Validation accuracy	Macro-F1
Primary category	62.88%	64.95%
Secondary category	95.50%	69.22%
Urgency	97.50%	92.64%

Primary categorization remains the main quality limitation. Secondary accuracy alone can conceal weak performance on uncommon secondary labels; macro-F1 therefore accompanies accuracy. Confidence below 0.5 triggers a human-review recommendation. Confidence is the model probability for the returned primary label, including after any rule override; it is not independently calibrated.

Automated API tests exercise authorization on every prediction/job route, input errors, output structure, issue precedence, ordering, idempotency, job status, pagination, and deletion. Desktop/mobile browser checks exercise automatic server access, classification, batch filtering/downloads, counters, email fields, async jobs, and cancellation. A local 2,001-ticket job passed submission, polling, complete ordered results, and deletion using the configured key. Another local check passed 64 rounds of valid-key, invalid-key, and health requests with eight concurrent workers. These checks verify implementation behavior; they do not prove compliance with the missing official YAML and JSON Schemas or operation on a public host.

Saved-artifact evaluation gives primary-category accuracy of 53.93% for English (280 examples), 45.62% for Sinhala (160), 90.00% for Singlish (200), 62.50% for Tamil (120), 65.00% for Tanglish (20), and 55.00% for mixed text (20). Small groups have substantial uncertainty. The largest confusion is missing/wrong orders classified as payment/refund (33 examples), followed by delivery delay as payment/refund (13). Better native-script training examples and multi-issue separation are priorities for future experiments; no improvement is claimed without a measured comparison.

5. Submission and operational plan

Before submission: validate every official request/response schema, compare routes/status/error formats with the supplied OpenAPI contract, build and run the Docker image with network disabled, deploy the authenticated service with persistent job storage and the exact evaluation key, test through the public URL, publish the repository, record the 15-minute walkthrough as an unlisted YouTube video, and export this report to a maximum-five-page PDF. Use `docs/submission-checklist.md` to record evidence and final URLs. No public-host or independent-holdout claims should be made until verified.